Lookking Defense Q&A Report

Student: Karim Akoudad

Project: Lookking - Multi-agent AI assistant for places and business leads

Date: May 17, 2026

Purpose: Short defense preparation report with answers to likely professor questions.

1. Executive Summary

Lookking is a Telegram chatbot that helps users find either real places or potential business leads. The user starts the bot, chooses a mode, writes a natural-language request, and receives the top ranked results. For places, the bot uses live OpenStreetMap data and returns clickable Google Maps/OpenStreetMap links. For leads, it searches a local business leads dataset. A trained deep learning model scores how well each candidate matches the user query.

The system combines:

- A Telegram user interface.
- A CrewAI multi-agent workflow.
- A fine-tuned DistilBERT matching model.
- Live OpenStreetMap/Nominatim search.
- CSV datasets for fallback places, leads, and model training.
- Human-in-the-loop steps through mode selection and result refinement.

2. What Does The App Do?

Lookking solves two practical search problems:

1. Find a place: restaurants, spas, gyms, hotels, cafes, barbershops, etc.
2. Find leads: businesses that could become clients for a freelancer or agency.

Example user flows:

- Place mode: `sushi restaurant Rabat open now`
- Leads mode: `I offer video editing for restaurants in Rabat`
- Refinement: `but cheaper`, `in Casablanca`, `with parking`

The app returns ranked results with explanations. For places, it also includes live map links so the result can be verified.

3. Where Is The Project? Google Colab, VS Code, Or Something Else?

The project is implemented as a local Python application, not as a Google Colab notebook.

Project location:

```
/Users/karimakoudad/Desktop/akoudad/0.1 FILES/04. apps and dev/lookking
```

Development environment:

- Local machine.
- Python virtual environment: `venv/`.
- Code can be opened in VS Code or any editor.
- Runtime entry point: `main.py`.

Google services are only used for the Gemini API key through Google AI Studio. Google Colab is not required because the training and demo can run locally. The trained model is already saved as:

```
model/lookking_model.pt
```

If asked why not Colab:

> I did not need Colab because the dataset is small and the DistilBERT fine-tuning can run locally. Keeping everything local also makes the Telegram bot, model file, data, and code easier to demonstrate as one reproducible application.

4. What Are The Agents?

The project uses three CrewAI agents defined in `agents/crew_setup.py`.

Agent 1: Query Orchestrator

Role:

- Understand the user's raw message.
- Detect whether the user wants places or leads.
- Extract city, category, and important constraints.

Example:

Input:

```
[MODE: places] sushi restaurant Rabat open now
```

Output concept:

```
mode = places
city = Rabat
```

```
category = sushi restaurant
criteria = open now
```

Agent 2: Discovery Specialist

Role:

- Search for candidate results.
- Uses `Search Places` for places.
- Uses `Search Leads` for business leads.

Tools:

- `search_places`
- `search_leads`

For places, it tries live OpenStreetMap/Nominatim first. If live search fails or returns nothing, it falls back to the local CSV.

Agent 3: AI Match Scorer

Role:

- Score every candidate against the user's query.
- Uses the trained deep learning model.
- Ranks the candidates and returns the top 3.

Tool:

- `score_match`

The scorer outputs:

- Match label: Low, Medium, or High.
- Confidence percentage.
- Explanation of why the candidate matches.

5. How Does The Pipeline Work?

The pipeline is sequential:

```
Telegram message
  -> Query Orchestrator
  -> Discovery Specialist
  -> AI Match Scorer
  -> Telegram response
```

CrewAI passes context from one task to the next. The discovery agent uses the intent extracted by the orchestrator. The scoring agent uses both the original query and the discovered candidates.

This design is better than one large prompt because each agent has a clear responsibility.

6. How Did You Train The Model?

The trained model is a query-candidate matching classifier.

Training script:

```
model/train.py
```

Training data:

```
data/training_data.csv
```

Dataset size:

- 295 query-candidate pairs.
- 145 high matches.
- 75 medium matches.
- 75 low matches.

Training method:

1. Generate labeled examples from the project domain.
2. Format each sample as:

```text
[QUERY]: user query [SEP] [CANDIDATE]: candidate description
```

3. Tokenize with `DistilBertTokenizerFast`.
4. Split into train/test sets using stratification.
5. Fine-tune the last part of DistilBERT and a custom classifier head.
6. Save the best model to `model/lookking_model.pt`.
7. Save metrics to `model/metrics.json`.
8. Save a confusion matrix image to `model/confusion_matrix.png`.

Training settings:

- Batch size: 16.
- Epochs: 12.
- Max sequence length: 128.

- Optimizer: AdamW.
- Loss: CrossEntropyLoss.
- Test split: 20%.

Final recorded metric:

- Accuracy: 100% on the synthetic test split.

Important honest explanation:

> The accuracy is high because the dataset is synthetic and controlled. It proves the training pipeline works, but it is not the same as production-level validation on noisy real-world data.

7. What Models Did You Use?

Deep Learning Model

Model:

```
distilbert-base-uncased
```

Use:

- Query-candidate relevance classification.
- Predicts Low Match, Medium Match, or High Match.

Architecture:

- DistilBERT encoder.
- Embeddings frozen.
- First 4 transformer layers frozen.
- Last 2 transformer layers trainable.
- Custom MLP classification head:

- Linear 768 -> 256
- ReLU
- Dropout
- Linear 256 -> 64
- ReLU
- Dropout
- Linear 64 -> 3

LLM For Agents

Model currently configured:

```
gemini/gemini-1.5-flash
```

Use:

- Agent reasoning.
- Intent extraction.
- Candidate selection instructions.
- Final response organization.

The Gemini key is stored in `.env` as `GEMINI_API_KEY`. The key must not be shown in the report or presentation.

External Data Source

OpenStreetMap/Nominatim is used for live place search. It is not a model, but it gives real geographic data.

8. Did We Respect The Project Requirements?

I did not find the professor's original PDF inside the project folder, so this checklist is based on the available project report and the implemented code. If the exact PDF has extra rules, it should be checked separately.

```
| Requirement | Status | Evidence |
|---|---:|---|
| Real AI application | Done | Telegram bot solves place and lead search |
| Multi-agent system | Done | 3 CrewAI agents in `agents/crew_setup.py` |
| Deep learning model | Done | DistilBERT classifier in `model/model.py` |
| Model training | Done | `model/train.py`, `data/training_data.csv`, saved weights |
| External data/API | Done | OpenStreetMap/Nominatim in `tools/nominatim_tool.py` |
| Human-in-the-loop | Done | Mode picker and Add Info refinement |
| Working demo | Mostly done | Bot starts; final live query test still needed |
| Logs/traceability | Done | `logs/agent_logs.json` |
| Documentation | In progress | `PROJECT_REPORT.md`, this report |
| Final PDF report | Done by this file/PDF export | `LOOKKING_DEFENSE_QA_REPORT.pdf` |
| Slides | Not done yet | Need 10-15 slide deck |
| Demo video | Not done yet | Need screen recording |
| GitHub repo/README | Not done yet | No `.git` repo found in this folder |
```

Technical implementation is mostly complete. The remaining work is packaging for submission and defense: slides, demo video, GitHub/README, and final live test.

9. What Is Human-In-The-Loop In This Project?

Human-in-the-loop means the user is not passive. The user participates in key decisions:

1. The user chooses Place mode or Leads mode before search.
2. After results, the user can click Done or Add Info.
3. If the user chooses Add Info, the bot merges the new constraint with the previous query and reruns the pipeline.

Why this matters:

- It reduces ambiguity.

- It lets the user correct the search without starting from zero.
- It makes the system more reliable and more controllable.

10. Why Did You Use CrewAI?

CrewAI was used because the project is naturally split into specialized roles:

- One agent understands the request.
- One agent searches.
- One agent scores and ranks.

This makes the system easier to explain, debug, and extend. For example, a future version could add a fourth agent for saving approved results or generating outreach messages.

11. Why Did You Use DistilBERT Instead Of Only Gemini?

Gemini is used for reasoning and orchestration, but the project also needs a real trained deep learning component.

DistilBERT is useful because:

- It is smaller and faster than BERT.
- It understands text similarity and semantic matching.
- It can be fine-tuned on our own labeled data.
- It gives a clear classification output: Low, Medium, High.

This shows that the project is not only API prompting; it includes a trained model.

12. Why OpenStreetMap/Nominatim?

OpenStreetMap/Nominatim was chosen because:

- It is free.
- It does not require a paid API key.
- It gives real-world place data.
- It supports Morocco search with `countrycodes=ma`.
- It makes demo results verifiable through map links.

Limitation:

Nominatim does not provide real ratings or prices. The app creates synthetic rating and price values from the OSM importance score. In production, a review API such as Google Places or Yelp would be better.

13. What Happens If An API Fails?

If Nominatim fails or returns no place result:

- The place search falls back to `data/places.csv`.

If Gemini quota fails:

- The bot returns a system error.
- Practical fix: use a fresh Google AI Studio key or reduce the number of LLM calls.

If the model file is missing:

- `main.py` checks setup and tells the user to train the model.

14. What Are The Main Files?

File	Purpose
`main.py`	Checks setup and starts Telegram bot
`bot/telegram_bot.py`	Telegram UI, state machine, HITL buttons
`agents/crew_setup.py`	CrewAI agents, tasks, and LLM config
`tools/search_tool.py`	Search tools for places and leads
`tools/nominatim_tool.py`	Live OpenStreetMap/Nominatim search
`tools/dl_scorer_tool.py`	Deep learning scoring tool
`model/model.py`	DistilBERT model architecture
`model/train.py`	Training script
`data/training_data.csv`	Training dataset
`data/places.csv`	Synthetic fallback place data
`data/leads.csv`	Synthetic lead data
`model/lookking_model.pt`	Saved trained model
`model/metrics.json`	Evaluation metrics

15. Questions The Professor May Ask

Q1. Explain your app in one minute.

Lookking is a Telegram AI assistant for finding places and business leads. A user chooses Place or Leads mode, writes a natural-language query, and the system uses three agents to understand the request, search candidates, and rank the best results. For places, it uses live OpenStreetMap data. For ranking, it uses our own trained DistilBERT matching model. The result is a top-3 list with explanations and map links.

Q2. Is this just ChatGPT/Gemini?

No. Gemini is only used for agent reasoning. The project also includes a trained deep learning model, local datasets, tools, live external API integration, Telegram UI, and human-in-the-loop interactions.

Q3. What is the deep learning contribution?

The deep learning contribution is a fine-tuned DistilBERT classifier that scores how well a candidate result matches a user query. It is trained on 295 labeled query-candidate pairs and predicts Low, Medium, or High match.

Q4. Why is your accuracy 100%?

Because the test data is synthetic and follows clear patterns. It proves that the pipeline works, but I would not claim production-level accuracy. For production, I would collect real user feedback and evaluate on real-world data.

Q5. How do the agents communicate?

The CrewAI process is sequential. The first task produces the structured intent, the second task uses that context to search, and the third task uses the candidate list to score and rank.

Q6. What if the user writes a vague query?

The mode picker already removes one big ambiguity: places vs leads. If the query is still vague, the agents try to infer the category and criteria. The user can then click Add Info to refine the results.

Q7. How would you improve it?

I would add real business lead APIs, persistent storage with SQLite, multilingual support for French/Arabic/Darija, caching for Nominatim, unit tests, and a better dataset collected from real user interactions.

Q8. What are the limitations?

The lead data is synthetic. Ratings and prices from OSM are synthetic because OSM does not provide review scores. Gemini free tier can hit quota. The model was trained on a small synthetic dataset. There is no GitHub repo or slides yet.

Q9. How do you run the app?

From the project folder:

```
source venv/bin/activate
python3 main.py
```

Then open Telegram and send `/start` to the bot.

Q10. What is left before final submission?

Stop any old bot process, run one final live test, create slides, record a demo video, create/push a GitHub repo, and submit the final PDF report.

16. Current Status

Done:

- Telegram bot flow.
- Multi-agent pipeline.
- Gemini LLM setup.
- Deep learning model training and inference.
- Live OpenStreetMap search.
- CSV fallback data.
- Progress and defense reports.

Not done yet:

- Final live Telegram query test after restarting the bot.
- Slide deck.
- Demo video.
- GitHub repository and README polish.
- Exact comparison against professor PDF, because the PDF was not found in this workspace.

17. Recommended Final Defense Message

The strongest way to present the project is:

> Lookking is not only a chatbot. It is a complete AI system with a Telegram interface, three specialized agents, a trained DistilBERT model, live map data, and human-in-the-loop refinement. The LLM handles reasoning, the tools handle data access, and the deep learning model handles relevance scoring.